

# Code Review: Mqtt\_Json\_Format\_V3.0\_Dual\_Broker

Date: 29 September 2026 | Repository: REConnect\_Firmware\_Modules (commit c5e8d46)

## Scope

The new dual-broker code in `src/` was compared against the previous primary/secondary failover implementation in `src_original/`. The review focused on correctness: protocol compatibility with the server, thread safety, and message delivery. Findings 1-6, 8 and 11 were confirmed directly in the source; findings 7, 9 and 10 are based on reading the call paths and should be confirmed during the fix.

## Key takeaway

The rename of **primary/secondary** to **mqtt1/mqtt2** also changed Redis field names and server command strings, not just variable names (findings 1-3). This breaks MQTT broker configuration from the server and should be fixed first. Findings 4-6 can cause lost acknowledgements, a permanent hang, or a crash.

## Summary

| #  | Severity | Location                                                                    | Issue                                                               |
|----|----------|-----------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1  | High     | <code>src/set_cfg_request.c:34</code>                                       | Set-config looks up non-existent Redis field "mqtt1"                |
| 2  | High     | <code>src/set_cfg_request.c:1343</code>                                     | DATA_TYPE strings no longer match what the server sends             |
| 3  | Medium   | <code>src/get_cfg_request.c:71</code>                                       | Get-config reports wrong field and wrong JSON key                   |
| 4  | High     | <code>src/mqtt_connect.c:1360</code><br>( <code>mqtt_send_msg</code> )      | Command ACKs published to the wrong topic                           |
| 5  | High     | <code>src/mqtt_connect.c:1382</code> and<br><code>:1216</code>              | Unbounded wait for publish completion can hang the unit             |
| 6  | High     | <code>src/mqtt_connect.c:3357</code><br>( <code>update_mqtt_time</code> )   | Redis accessed from Paho callback threads                           |
| 7  | Medium   | <code>src/mqtt_connect.c:3384</code>                                        | <code>last_message_time</code> updated for both brokers on any send |
| 8  | Medium   | <code>src/main.c:1369</code> (worker loop)                                  | Cloud LED never turns on                                            |
| 9  | Medium   | <code>src/mqtt_connect.c:2809</code><br>( <code>on_message_arrived</code> ) | Commands from two brokers share one buffer slot                     |
| 10 | Medium   | <code>src/mqtt_connect.c:2575</code>                                        | FetchDay responses can go to the wrong broker                       |
| 11 | Medium   | <code>src/modbus_json_export.c:1039</code>                                  | Device count and export loop disagree, producing invalid JSON       |

High: 5 Medium: 6

## Detailed findings

### 1. Set-config looks up non-existent Redis field "mqtt1" [High]

**Location:** `src/set_cfg_request.c:34`

**Problem:** The broker lookup loop calls `rget_int(ctx, tmp, "mqtt1", -1)` on each `mqtt_N_cfg` hash. The field stored in Redis is "primary" (still read that way at `src/main.c:1571`).

**Impact:** `rget_int` returns -1, hash stays empty and `set_mqtt_cfg` returns -1. Every MQTT broker set-config command fails.

**Suggested fix:** Restore the Redis field name "primary". Only rename variables, never Redis keys/fields or protocol strings.

## **2. DATA\_TYPE strings no longer match what the server sends [High]**

**Location:** src/set\_cfg\_request.c:1343

**Problem:** set\_cfg\_export\_json compares against "MQTT\_BROKER\_mqtt1" / "MQTT\_BROKER\_mqtt2". The server sends "MQTT\_BROKER\_PRIMARY" / "MQTT\_BROKER\_SECONDARY" (Json\_Format\_Set\_Config.txt lines 6 and 25).

**Impact:** MQTT broker set-config commands from the server match nothing and are ignored.

**Suggested fix:** Restore "MQTT\_BROKER\_PRIMARY" and "MQTT\_BROKER\_SECONDARY".

## **3. Get-config reports wrong field and wrong JSON key [Medium]**

**Location:** src/get\_cfg\_request.c:71

**Problem:** ADD\_BOOL("mqtt1", "mqtt1") reads a Redis field that does not exist and emits JSON key "mqtt1" instead of "PRIMARY".

**Impact:** Response always contains "mqtt1": "NO"; the server never receives the PRIMARY key.

**Suggested fix:** Change back to ADD\_BOOL("primary", "PRIMARY").

## **4. Command ACKs published to the wrong topic [High]**

**Location:** src/mqtt\_connect.c:1360 (mqtt\_send\_msg)

**Problem:** The CMD\_ACK\_TOPIC branch in mqtt\_send\_msg is commented out (line 1281), but ACKs are still sent with CMD\_ACK\_TOPIC (lines 2498-2523). They fall through to the else branch.

**Impact:** ACKs go to cmd\_response\_topic instead of PUB\_ACK\_TOPIC/<serial>; the server never receives command acknowledgements.

**Suggested fix:** Restore the CMD\_ACK\_TOPIC branch so ACKs publish to the ack topic.

## **5. Unbounded wait for publish completion can hang the unit [High]**

**Location:** src/mqtt\_connect.c:1382 and :1216

**Problem:** After MQTTAsync\_sendMessage, the worker blocks on pthread\_cond\_wait with no timeout. If the connection drops after the send is accepted, a QoS 1/2 message may never get onSuccess/onFailure (e.g. clean\_session=0, Paho keeps it for resend).

**Impact:** The worker thread is the only thread that reconnects, destroys clients and processes commands, so the DCU hangs permanently for both brokers.

**Suggested fix:** Use pthread\_cond\_timedwait (e.g. a few seconds beyond the publish timeout) and treat a timeout as a failed publish.

## **6. Redis accessed from Paho callback threads [High]**

**Location:** src/mqtt\_connect.c:3357 (update\_mqtt\_time)

**Problem:** update\_mqtt\_time now calls redisCommand(ctx, ...) directly. It is called from Paho callback threads: on\_connect\_success (lines 257, 305), connect failure (498) and connectionLost (768). The worker thread uses the same hiredis context at the same time.

**Impact:** hiredis contexts are not thread-safe. A reconnect during a publish or Redis read can corrupt the connection or crash the process.

**Suggested fix:** Go back to the earlier design: callbacks only set a flag (e.g. mqtt\_time\_update\_req) and the worker does the Redis write. Alternatively, give the callbacks their own Redis context or guard the context with a mutex.

## **7. last\_message\_time updated for both brokers on any send [Medium]**

**Location:** src/mqtt\_connect.c:3384

**Problem:** update\_mqtt\_time(1) is called after every successful mqtt\_send\_msg and writes last\_message\_time to both mqtt1\_status\_hash and mqtt2\_status\_hash when both are connected, regardless of which broker sent the message.

**Impact:** Status is misleading, e.g. a health-check sent only to mqtt1 also advances mqtt2's time.

**Suggested fix:** Pass the sending connection into update\_mqtt\_time and update only that broker's hash.

### **8. Cloud LED never turns on [Medium]**

**Location:** src/main.c:1369 (worker loop)

**Problem:** The worker loop no longer calls `mqtt_led_set(mqtt_led_connected)`; the old calls are commented out (main.c:758-760). The only remaining call is `mqtt_led_set(0)` in `mqtt_led_init`.

**Impact:** The cloud LED on GPIO87 stays off even when both brokers are connected.

**Suggested fix:** Restore the LED update in the worker loop (on if at least one broker is connected).

### **9. Commands from two brokers share one buffer slot [Medium]**

**Location:** src/mqtt\_connect.c:2809 (on\_message\_arrived)

**Problem:** Both brokers stay subscribed, but `on_message_arrived` writes into a single `mqtt_cmd_buffer / mqtt_cmd_recv` slot. The worker drains it only once per loop (`sleep(3)` plus long publish phases).

**Impact:** If `mqtt1` and `mqtt2` both send a command before the worker reads it, the first command is overwritten and lost silently; `mqtt_cmd_broker` names only the second broker.

**Suggested fix:** Use a small mutex-protected queue (or one slot per broker) and record the source broker with each command.

### **10. FetchDay responses can go to the wrong broker [Medium]**

**Location:** src/mqtt\_connect.c:2575

**Problem:** `Fetchday_cmd_broker` is a single global overwritten by every FetchDay request, while responses are queued in the shared Redis list `mqtt_command_resp` (drained at main.c:968-975).

**Impact:** If `mqtt2` sends a FetchDay before `mqtt1`'s responses are drained, the rest of `mqtt1`'s responses are published to `mqtt2` and `mqtt1` never receives them.

**Suggested fix:** Store the target broker with each queued response entry (or use one response list per broker).

### **11. Device count and export loop disagree, producing invalid JSON [Medium]**

**Location:** src/modbus\_json\_export.c:1039

**Problem:** The counting loop (line 991) counts RTU devices `0..MAX_RTU_DEVICES-1` on every port. The export loop iterates `0..MAX_MODRTU_DEVICES-1` and only on ports with `device_type == 2`. `is_last` is computed against the wrong total.

**Impact:** Example: port 0 has devices 0 and 3 enabled, no TCP devices: `total=1`, device 0 is written as last (no comma), device 3 is appended directly after it and the Modbus JSON contains `}{`. If a port is skipped, the last device never gets `is_last`.

**Suggested fix:** Use the same port filter and device limit in both loops, or emit the separator before each element after the first instead of pre-computing `is_last`.

## **Recommended order of work**

1. Restore Redis field names and server strings (findings 1-3), then test `set-config` and `get-config` for both brokers against the server.
2. Restore the ACK topic branch (4) and verify ACKs arrive on `PUB_ACK_TOPIC/<serial>`.
3. Fix thread safety and the blocking wait (5, 6); test by dropping the network during publish.
4. Fix the dual-broker bookkeeping (7, 9, 10), the LED (8) and the Modbus JSON export (11).